

UXL SKILLS EVALUATOR

oneDPL Skill

Execution policies, data, iterators, synchronization, and ordering

September 2026 • 4 of 6 evaluation tasks implemented

Why this is useful to oneDPL at first glance

- **Choose host versus device on purpose** — Execution policy follows data location, queue ownership, and workload — not an assumption that GPU is always faster.
- **Preserve the standard algorithm contract** — Stability, ordering, iterator requirements, and exception expectations stay visible.
- **Catch asynchronous lifetime bugs** — Agents must synchronize before host reads and keep USM/buffer ownership valid.
- **Make portability claims honest** — Algorithm/device/backend support is checked against current project sources.

What the skill tells an agent today

- **Start from a serial reference** — Preserve std behavior before changing policy or data placement.
- **Describe data and iterators** — Host memory, USM, buffers, wrapper types, iterator category, lifetime, and ordering.
- **Use caller-owned queues** — Construct device policies from the application's queue when selection and dependencies already exist.
- **Synchronize explicitly** — Device completion must be proven before results are consumed on the host.
- **Benchmark end to end** — Include transfers, queue synchronization, launch cost, and representative ranges.

Evaluation inventory: 4 implemented, 2 device gaps

- **Implemented** — Missing device synchronization; iterator-category failure; stable-ordering contract; move-only numeric accumulator.
- **Incident evidence** — Iterator-category and move-only-accumulator tasks are grounded in maintainer incidents.
- **Planned** — Host-or-device sort choice and transfer-inclusive benchmark on declared target hardware.
- **What this proves** — Current tasks check contracts and repair behavior; they do not mirror the full algorithm support table.

How we chose the content

- **Official sources** — Execution-policy docs, device-policy construction, examples, tests, releases, and known issues.
- **Standard-library boundary failures** — Iterator categories, stable ordering, move-only types, and device lambda restrictions.
- **SYCL execution failures** — Queue ownership, synchronization, lifetime, device selection, and data movement.
- **Deliberate gaps** — A real target lane is required before host/device or transfer-performance claims become evidence.

What project ownership would look like

- **Review policy and algorithm guidance** — Especially supported policies, algorithm contracts, iterator restrictions, and queue construction.
- **Keep the boundary with SYCL clear** — Project guidance owns oneDPL behavior; shared SYCL skill owns general toolchain/runtime triage.
- **Provide real device scenarios** — Help choose the first host/device decision and transfer-inclusive task worth maintaining.
- **UXL keeps the machinery** — Common validators, Harbor, dashboards, and vendor-neutral runner contracts remain shared.

A focused 30-minute maintainer review

- **1. Confirm the trigger** — When should an agent use oneDPL guidance versus std, oneTBB, or direct SYCL?
- **2. Correct policy gotchas** — Add one unsupported combination or common lifetime/synchronization failure.
- **3. Judge the six tasks** — Keep, rewrite, or remove; prioritize one credible device case.
- **4. Name an owner** — A reviewer for release-time support changes and task authenticity.
- **Outcome** — Agents arrive with policy, data, iterator, queue, synchronization, and evidence contracts already stated.